ZSH, PLUGINS E O PROMPT DO KALI

*Quatro abas abertas: duas locais, uma no laboratório, uma no servidor de produção que ninguém deveria estar tocando. O prompt das quatro dizia `$`. A pessoa digitou `systemctl stop nginx`, apertou Enter — e **só então** olhou o título da janela. **O prompt não é enfeite: é a instrumentação permanente da sessão.***

093
GUIAS LINUX

ANATOMIA DO PROMPT: A LINHA MAIS LIDA DO TERMINAL

```
kali @ kali : ~/lab-zsh $  … o comando que você digita …        1  12:04
```

- **`%n` usuário** — quem você é (vira `root` sob `sudo -i`: é o primeiro campo a conferir antes de um comando destrutivo.
- **`%~` diretório atual**, com o home como til. `%(4~|.../%3~|%~)` trunca caminho longo.
- **`%m` máquina** — nome curto do host. É o campo que teria evitado o `systemctl stop` na máquina errada.
- **`%#` privilégio** — `%` para usuário comum, `#` para root. Sinal de que dali em diante não há rede de proteção.
- **`RPROMPT` à direita, e some sozinho** quando o comando cresce: `%?` (código de saída) e `%*` (hora). O `%(?..)` só o mostra quando algo falhou.

Dois detalhes decidem se o prompt funciona. 1. `setopt prompt_subst` reavalia a expansão a cada redesenho — sem ele, o código de saída e o ramo do Git *congelam* no valor que tinham quando o `.zshrc` foi lido. **2.** Todo byte não imprimível precisa ser marcado como tal: `%{ %}` no Zsh, `\[ \]` no Bash. Sem ela o shell conta os bytes invisíveis da cor como largura, erra o cursor e **a linha se sobrescreve** quando você navega no histórico — a causa nº 1 do "meu prompt embaralha na seta para cima". Com `%F{cor}…%f` em vez de escape cru, o Zsh cuida disso sozinho.

O QUE O ZSH ACRESCENTA — E QUANTO CUSTA CADA CAMINHO

COMPLETAÇÃO COM MENU

O `compsys` lê a página de manual: entende subcomando, navega com as setas e descreve cada candidato. `autoload -Uz compinit && compinit` + `zstyle ':completion:*' menu select`.

QUALIFICADORES DE GLOB

`**/*.log(.)` só arquivos regulares · `(.mh-2)` últimas 2 h · `(.Lm+10)` maiores que 10 MB · `*(/)` só diretórios. Substitui muito `find` **no uso interativo**; em script, `find` continua a escolha portável.

HISTÓRICO DE VERDADE COMPARTILHADO

`setopt share_history` — o comando digitado numa aba já está na seta-para-cima de outra aba **aberta**. Junto: `hist_ignore_all_dups`, `hist_ignore_space`, `hist_verify`, `extended_history`.

OS DOIS PLUGINS QUE VALEM SOZINHOS

`zsh-autosuggestions` completa em cinza pelo histórico (→ aceita). `zsh-syntax-highlighting` colore **enquanto você digita**: comando inexistente em vermelho *antes* do Enter.

CAMINHO	O QUE DÁ · O QUE CUSTA
o .zshrc do Kali	prompt, menu, histórico e os 2 plugins · **nada: é o padrão**
2 plugins por apt	autosuggestions + highlighting · duas linhas de `source`
zinit / antidote	gerenciador com carga preguiçosa · aprender a ferramenta
Oh My Zsh	ecossistema pronto · **inicialização lenta**, muito que não se usa
starship	um prompt só para Bash, Zsh e Fish · binário externo

Meça antes de otimizar: `time zsh -i -c exit`. Abaixo de **200 ms** ninguém percebe; acima de **500 ms** incomoda, multiplicado por dezenas de aberturas por dia. O culpado costuma ser `compinit` sem cache — ou um prompt que roda `git status` a cada Enter, o que trava o terminal em repositório grande.

NA PRÁTICA — O PROMPT DO KALI E A MIGRAÇÃO DO BASH

kali@kali: ~ — ~/.zshrc

```
# o Kali nao usa framework: o prompt vem do
# pacote kali-defaults, ligado por uma variavel
PROMPT_ALTERNATIVE=twoline   # oneline | backtrack
NEWLINE_BEFORE_PROMPT=yes
# prompt proprio, com codigo de saida a direita
setopt prompt_subst
PROMPT='%F{cyan}%n%f@%F{green}%m%f:%F{yellow}%~%f %# '
RPROMPT='%(?..%F{red}[%?]%f )%F{8}%*%f'
# a ORDEM importa: highlighting SEMPRE por ultimo
source /usr/share/zsh-autosuggestions/…zsh
source /usr/share/zsh-syntax-highlighting/…zsh
$ exec zsh   # recarrega sem abrir processo novo
```

VARIANTE / ARMADILHA	O QUE MUDA DE FATO
`twoline`	padrão do Kali: o comando **sempre começa na coluna 1** — caminho longo não empurra o cursor. As outras: `oneline` e `backtrack`
migrar do Bash	o `.bashrc` não é lido: alias e função vão **movidos**; `shopt` vira `setopt` e array começa em **1**, não em 0
Nerd Font	o ícone de ramo/pasta só existe nela — e instala-se no **cliente**, não no servidor

O `zsh-syntax-highlighting` tem de ser o último `source` do arquivo. Ele instala ganchos no widget de edição de linha do Zsh, e qualquer plugin carregado *depois* pode substituí-los, desligando o realce **em silêncio**: não há mensagem de erro, a cor simplesmente não aparece e você passa meia hora procurando um pacote que já estava instalado. **E o shell interativo não afeta script: o shebang manda.** Um arquivo com `#!/bin/bash` roda em Bash mesmo que o seu terminal seja Zsh — desde que você não confie em nada do `.zshrc` lá dentro.

RESUMO RÁPIDO

- ✓ **O prompt é instrumentação, não decoração.** `%n %m %~ %#` respondem quem-você-é, onde-está e com-qual-privilégio — as três perguntas cuja resposta errada custa caro.
- ✓ **O código de saída no `RPROMPT` é a informação mais útil que o prompt padrão omite** — e o `%(?..)` só a mostra quando algo falhou.
- ✓ **Escape de cor não marcado embaralha a linha:** `%F{}` / `%f` no Zsh, `\[ \]` no Bash. E `prompt_subst`, ou o valor congela.
- ✓ **Dois plugins valem mais que qualquer framework** — e o `syntax-highlighting` sempre como **último** `source`, ou o realce some sem aviso.
- ✓ **Framework tem custo mensurável:** `time zsh -i -c exit`. Acima de 500 ms incomoda; o culpado é `compinit` sem cache ou prompt que consulta o Git a cada Enter.
- ✓ **O shebang manda no script.** Mas alias e função do `.bashrc` não migram sozinhos, e array no Zsh começa em 1.

NO KALI

O Zsh é padrão desde o Kali 2020.4, e a migração foi feita com cuidado: o `~/.zshrc` do `kali-defaults` já traz completação com menu, histórico compartilhado, os dois plugins essenciais e o prompt de duas linhas — você não precisa configurar *nada* para ter uma boa experiência. Vale lê-lo inteiro uma vez: é um exemplo bem escrito de configuração defensiva. **O `kali-tweaks` é o caminho oficial para trocar o prompt**, alterando `PROMPT_ALTERNATIVE` nos arquivos gerenciados pelo pacote; edição manual ali é sobrescrita na próxima atualização — o lugar seguro para o que é seu é o fim do seu `~/.zshrc`. **O prompt de root é visualmente distinto de propósito, e isso é decisão de segurança:** sob `sudo -i` o `%#` vira `#` e a cor muda. Num sistema em que quase toda ferramenta roda com privilégio, esse sinal permanente é a diferença entre apagar *o seu* diretório de testes e apagar *o do sistema* — não remova a distinção ao personalizar. **Em trabalho autorizado, informação vale mais que estética:** com várias sessões simultâneas — máquina local, pivô, alvo — `%m` e o código de saída no `RPROMPT` são o que impede confundir uma com a outra; pôr o nome do engajamento no prompt é o raciocínio de rastreabilidade da auditoria aplicado à própria sessão, e gravar tudo com `script` ou pelo `tmux` é hábito da casa. **Fora do Debian:** Zsh é Zsh (`dnf install zsh`); muda o caminho dos plugins — `/usr/share/zsh-<plugin>/` aqui, `/usr/share/zsh/site-functions/` no Fedora —, então dotfile portável testa `[[ -r caminho ]]` antes do `source`.